

语言程序设计

第十二讲 文件



张 华

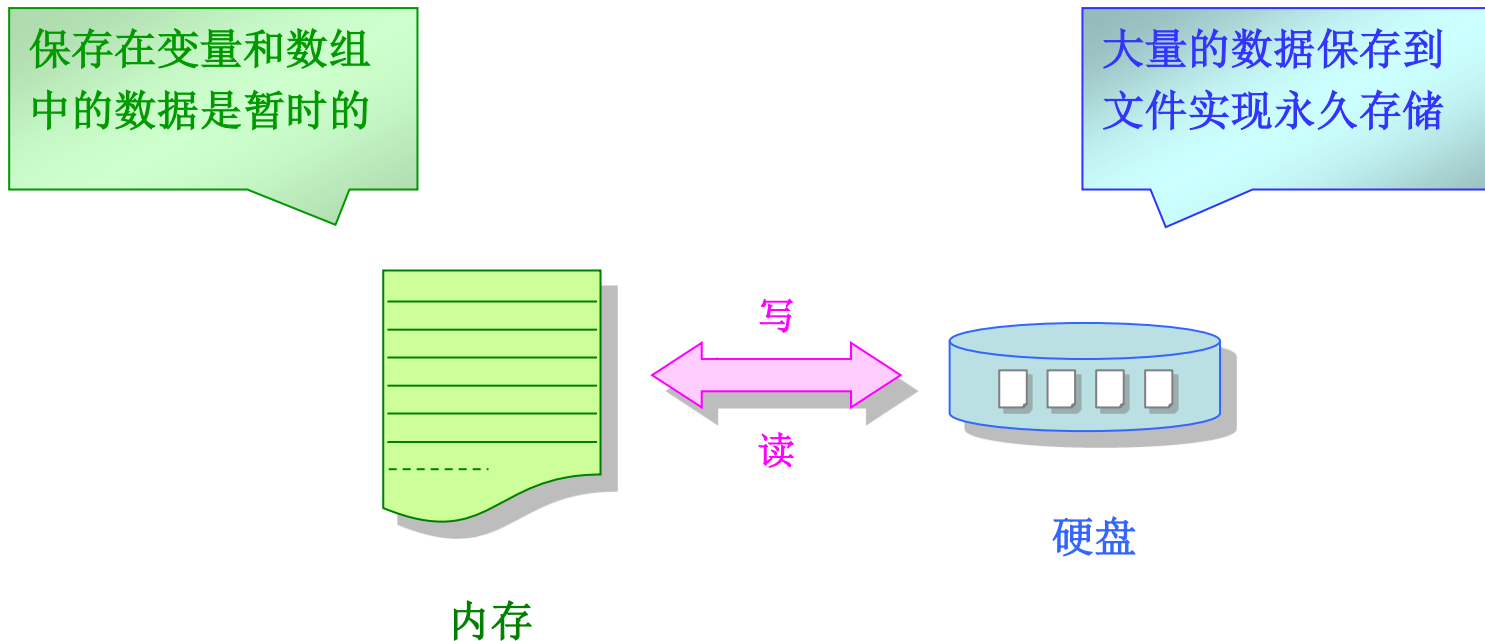
主要内容

- 文件的概念
- 文件指针
- 文件的打开与关闭
- 文件的读写
- 文件的定位
- 程序设计举例

简介

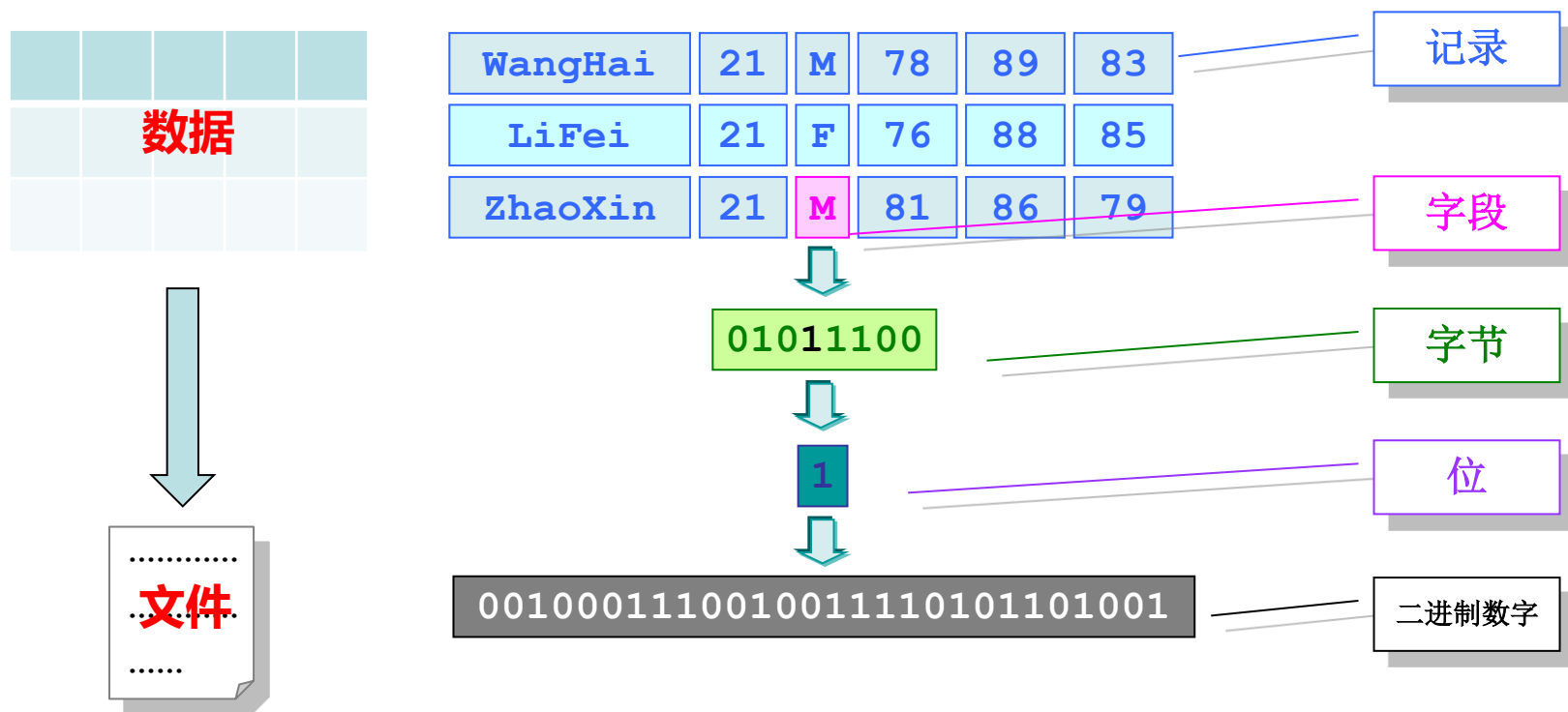
文件

✿ 存储在外存储器上的数据集合。



简介

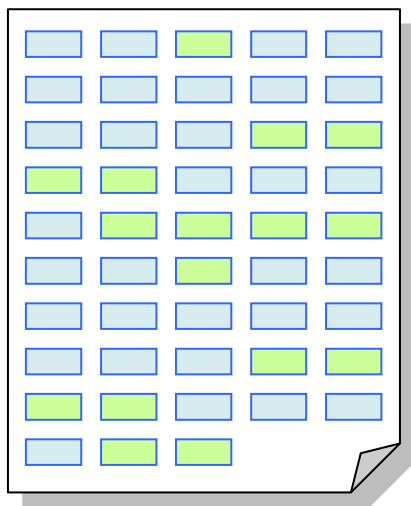
数据的层次结构



简介

C语言的文件处理

✿ C语言把文件看作是一个有序的字节或字符流。

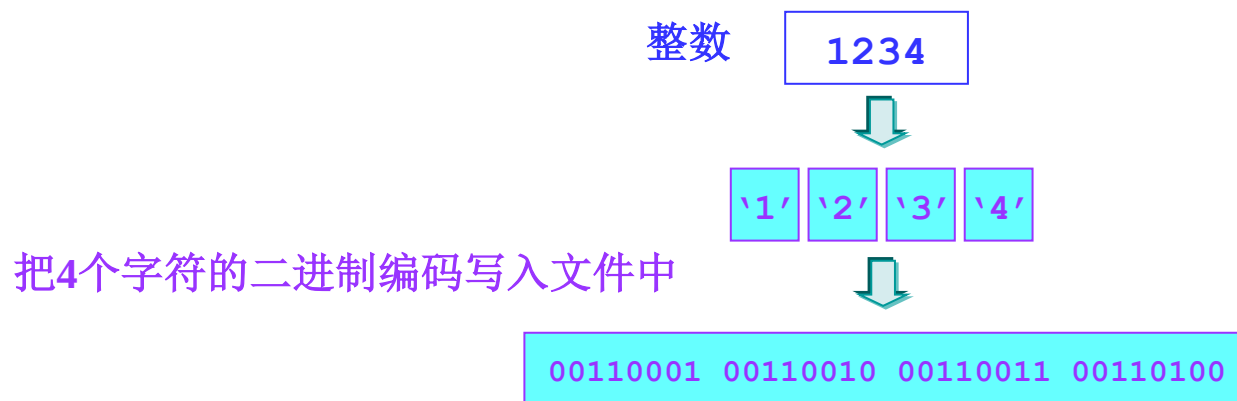


简介

文件的分类（按存储形式分）

✿ 文本文件

- 数据被作为字符，并以字符的二进制编码形式保存。
- 举例



简介

文件的分类（按存储形式分）

二进制文件

- 以数据在内存中的二进制形式保存。
- 举例

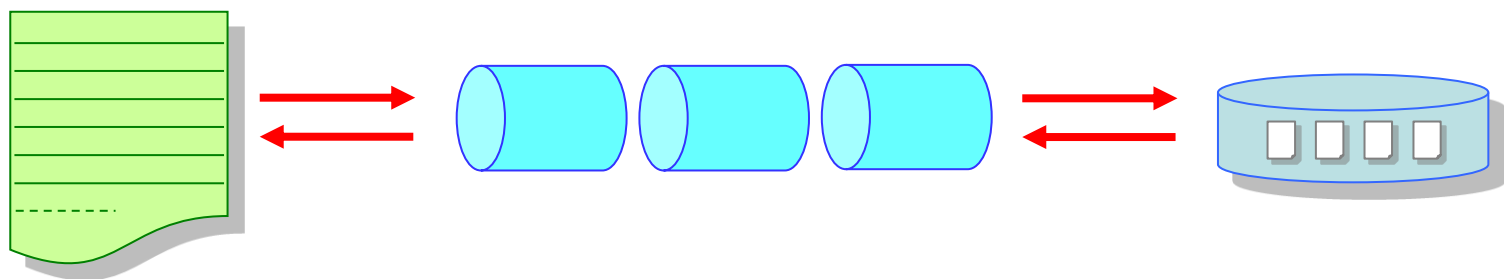


简介

文件的读写

缓冲文件系统

- 系统在内存中开辟一块缓冲区，以便与慢速的外设成块地进行数据交换。



- 向磁盘(文件)输出数据:
 - 数据→缓冲区，装满缓冲区后→磁盘文件。
- 从磁盘(文件)读入数据:
 - 先一次性从磁盘文件将一批数据输入到缓冲区，
 - 然后再从缓冲区逐个数据读入到变量。

文件指针

文件指针（指向文件的指针）

- 缓冲文件系统为每个使用的文件在内存中开辟一存储区存放其相关信息，这些信息用一个结构体变量保存，该结构体类型的名字是 **FILE**。
- 程序通过 **FILE** 类型指针建立和文件的联系，进行关于文件的一切操作。
- 定义形式：
FILE *指针变量名;
- 举例：

用于存放文件的缓冲区首地址，即指向一个文件。

```
FILE *fp;
```

➤ 定义了一个结构体 **FILE** 类型的指针变量：**fp**。

文件的打开

打开文件

✿ 方法:

- 为了打开一个文件，首先要为该文件定义一个文件类型指针；
- 然后用C语言提供的**fopen()**函数打开文件。
- **fopen**函数有两个参数：
 - “文件名”与“文件使用方式”。
 - 它们均是字符串。
- 一般形式:

```
FILE *文件指针名;
```

```
...
```

```
文件指针名=fopen("文件名","文件使用方式");
```

文件的打开

打开文件

文件使用方式

符号	意义
r (read)	以只读方式打开一个文本文件，若文件不存在，则返回空指针
w (write)	以只写方式打开或创建一个文本文件，并覆盖原有数据
a (append)	以追加方式打开或创建一个文本文件，并从文件末尾写数据
t (text)	文本文件，可省略不写
b (binary)	二进制文件
+	读和写

文件的打开

打开文件

✿ 举例

```
FILE *fp;  
fp = fopen("filea.dat", "r");
```

- 其意义是在当前目录下打开文件**filea.dat**，文件的使用方式为“只读”，并使**fp**指向该文件。

✿ 举例

```
FILE *fpabc;  
fpabc = fopen("c:\\fileabc.dat", "rb");
```

- 其意义是打开**c**盘根目录下的文件**fileabc.dat**，这是一个二进制文件，只允许按二进制方式进行读操作。

文件的关闭

关闭文件

✿ 使用 **fclose()** 函数。

✿ 一般形式：

fclose(文件指针);

✿ 举例

```
FILE *fp;  
...  
fclose(fp);
```

如果不关闭文件，将会丢失数据，并且一个C程序能同时打开的文件数有限，应该养成在使用完文件后关闭文件的习惯。

- 将 **fp** 指向的缓冲区中的数据存放到外存储器的文件中，然后释放该缓冲区，使文件指针变量不再指向与该文件对应的 **FILE** 结构，从而断开与文件的关联。
- 操作正常返回 **0**，否则返回 **EOF**（是在 **stdio.h** 文件中定义的符号常量，值为 **-1**）。

文件操作

文件操作的一般步骤

```
/* 定义声明文件指针 */  
FILE *fp;  
  
/* 打开文件 */  
fp = fopen("文件名", "r");  
  
/* 判断是否打开文件 */  
if (fp == NULL) {  
    /* 当文件打开失败时的处理 */  
}  
  
/* 对文件进行读写操作 */  
  
/* 关闭文件 */  
fclose(fp);
```

文件的读写

C语言支持两种读写文件的方式

顺序读写

- 从文件中一个接一个地读写数据，仅当读写了前一个数据后，才能读写下一个数据。
- 在读写数据后，文件位置指针自动跳到下一个读写位置。
- 标准输入输出只能进行顺序读写。

随机读写

- 从文件中读写数据时可不考虑数据在文件中的位置，可以在文件的任何位置读写。
- 在读写数据前，需要将文件位置指针移到读写位置。
- 当文件以读或写方式打开，文件位置指针设置在文件头，若文件以附加方式打开，位置设置在文件末尾。

字符的读写

读字符

✿ fgetc()函数

- 从指定文件读入一个字符，该文件必须是以读或读写方式打开。
- 返回值：
 - 调用成功时返回读入的字符；
 - 文件结束时返回文件结束符**EOF**。
- 举例

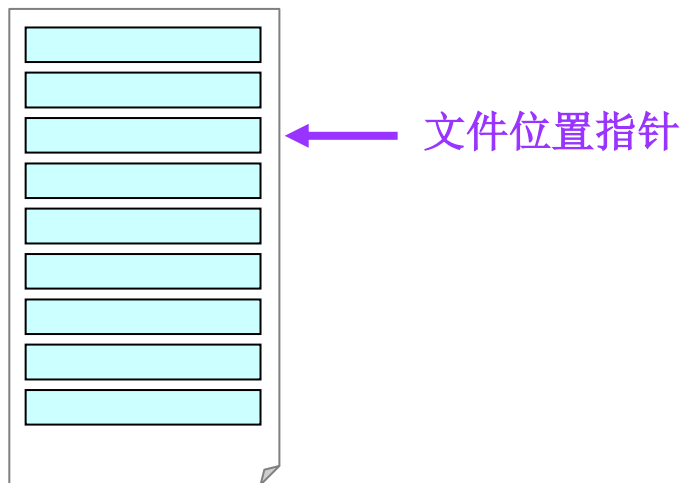
```
ch = fgetc(fp);
```

- 表示从文件指针变量**fp**指向的文件中读出一个字符，赋给字符变量**ch**

字符的读写

文件位置指针

- 文件型数据结构中，用文件位置指针指向当前对文件进行读写操作的位置。
 - 在文件打开时，该指针总是指向文件的第一个字节。读写文件时，每读写一个字符后，该位置指针的值会自动加1，指向下一个字符。
 - 改变位置指针的值，也就改变了下一次读写操作在文件中执行时的位置。



字符的读写

文件结束

问题

- 使用 **fgetc()** 函数，文件结束时返回文件结束符 **EOF**，其值为 **-1**。这在对文本文件操作时不会产生问题。但对二进制文件进行读操作时，由于 **-1** 是二进制数的合法值，故将影响文件数据的读取。
- 为解决此问题，**ANSI C** 提供了专门判断文件结束的函数 **feof()**。

feof() 函数

feof (文件指针) ;

- 判断文件位置指针是否已至文件尾
- 返回值：
 - 函数返回值为 **1** 时表示已至文件尾部；
 - 为 **0** 时则还未到文件结束处。

字符的读写

写字符

✿ fputc()函数

fputc (字符量, 文件指针);

➤ 将一个字符写到磁盘文件上去

➤ 返回值:

- 调用成功时返回输出的字符;
- 失败时返回**EOF**。

➤ 举例

```
fputc('a', fp);
```

- 表示向文件指针变量**fp**指向的文件写入一个字符' a'。

案例分析：字符的读写

问题

✿ 打开“c:\infile.c”文件，然后将其复制到“c:\outfile.c”。

源代码 (cw1601.c)

```
#include "stdio.h"
#include "stdlib.h"

void main()
{
    FILE *infp,*outfp;
    char  ch;

    if ((infp = fopen("c:\\infile.c", "r")) == NULL)
    {
        printf("Cannot open infile.c.\n");
        exit(0);
    }
```

案例分析：字符的读写

源代码

```
if ((outfp = fopen("c:\\outfile.c", "w")) == NULL)
{
    printf("Cannot open outfile.c.\n");
    exit(0);
}

while(!feof(infp))
    if ((ch = fgetc(infp)) != EOF)
        fputc(ch, outfp);

fclose(infp);
fclose(outfp);
}
```

字符串的读写

读字符串

✿ fgets()函数

fgets (字符数组名, n, 文件指针);

- 从指定文件读入n-1个字符到字符数组，最后加一个‘\0’结束符。在读完n-1个字符之前遇到换行符或文件结束符EOF，读入即结束。
- 返回值：字符数组的地址。
- 举例

```
fgets(str, n, fp);
```

- 从fp所指的文件中读出n-1个字符送入字符数组str中。

字符串的读写

写字符串

✿ fputs()函数

fputs (字符数组名, 文件指针);

- 向指定的文件输出一个字符串，‘\0’结束符不输出。其中字符串可以是字符串常量、字符数组名或字符指针。
- 返回值：
 - 调用成功时，返回0；
 - 否则返回EOF。
- 举例

```
fputs ("abcd", fp);
```

- 其意义是把字符串“abcd”写入fp所指的文件之中。

案例分析：字符串的读写

问题

- 从键盘上输入一串字符，写入文本文件“abc.c”中，再将文本文件的内容读出，显示在屏幕上。

源代码 (cw1602.c)

```
#include "stdio.h"
#include "stdlib.h"

void main()
{
    FILE *fp;
    char str[100], ch;

    if((fp = fopen("c:\\abc.c", "w")) == NULL)
    {
        printf("file open error.\n");
        exit(0);
    }
```

案例分析：字符串的读写

源代码

```
printf("请输入一串字符: \n");  
gets(str);  
  
fputs(str, fp);  
  
fclose(fp);  
  
if((fp=fopen("c:\\abc.c", "r")) == NULL)  
{  
    printf("file open error.\n");  
    exit(0);  
}  
  
while(!feof(fp))  
    if ((ch = fgetc(fp)) != EOF)  
        putchar(ch);  
  
fclose(fp);  
}
```

格式化读写

格式化读

✿ fscanf()函数

fscanf (文件指针, “格式控制字符串”, 地址表列);

- 从指定的文件中格式化读数据。
- 这个函数与格式输入函数scanf()很相似，它们的区别在于，scanf()函数是从键盘输入数据，而fscanf()函数是从文件读入数据，因此在fscanf()函数参数中多了一个文件指针，用于指出从哪个文件读入数据。
- 举例

```
fscanf(fp, "%f%d", &f, &d);
```

- 表示从磁盘文件中的读取一个浮点数和一个整数，分别存入变量f和d。

格式化读写

格式化写

✿ fprintf()函数

fprintf(文件指针, “格式控制字符串”, 输出表列);

- 格式化写数据到指定的文件中。
- 这个函数与**printf()**函数很相似，它们的区别在于，**printf()**函数是将数据输出到屏幕上，而**fprintf()**函数是将数据输出到文件中，因此在**fprintf()**函数参数中多了一个文件指针，用于指出将数据输出到哪个文件中。
- 举例

```
fprintf(fp, "f=%f, d=%d\n", f, d);
```

- 表示把变量**f**和**d**按指定的格式写入磁盘文件。

数据块的读写

以数据块方式读写文件

- ✱ `fscanf()`函数和`fprintf()`函数对磁盘文件读写，使用方便，易理解。但输入输出需要转化。
- ✱ 因此，在内存与磁盘频繁交换数据的情况下，最好用`fread()`和`fwrite()`函数。

`fread`(内存地址,数据项字节数,数据项个数,文件指针);

`fwrite`(内存地址,数据项字节数,数据项个数,文件指针);

➤ 通常用于对二进制文件的读写操作。

➤ 举例

```
int a[5];  
fread(a, sizeof(int), 5, fp);
```

– 表示从已打开的一个文件中读入5个整数，依次送入整型数组a中。

文件的随机读写

文件内部的位置

✿ 按要求移动位置指针，是实现随机读写的关键。

✿ rewind()函数

rewind(文件指针);

➤ 使位置指针重新返回到文件的开头，函数无返回值。

✿ ftell()函数

ftell();

➤ 得到当前文件指针位置。

➤ 返回值：

- 调用成功时，返回**long int** 数值，表示文件指针位置；
- 出错返回**-1L**。

文件的随机读写

文件内部的位置

✱ fseek()函数

fseek (文件指针, 位移量, 起始点);

➤ 改变文件的位置指针。

➤ 其中,

- 文件指针: 是文件打开时返回的文件指针。
- 位移量: 是指以起始点为基点, 向前向后移动的字节数, 可以为负值。大多数C语言版本要求位移量为**long**型数。
- 起始点: 表示从何处开始计算位移量, 规定的起始点有:

起始点	名字	数字代号
文件开始	SEEK_SET	0
当前位置	SEEK_CUR	1
文件末尾	SEEK_END	2

➤ 返回值: 操作成功返回**0**, 否则返回非**0**。

案例分析：文件的随机读写

问题

- 设文件“**alphabet.c**”中存放了字母表“A…Z”，现在打开这个文件，用从尾部倒着读的方式将其信息读出并送屏幕显示。

源代码 (cw1603.c)

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    FILE *fp;
    long i;
    if((fp=fopen("c:\\alphabet.c", "rb"))== NULL) {
        printf("file open error.\n");
        exit(0);
    }
```

案例分析：文件的随机读写

源代码

```
for(i=1;i<=26;i++) {  
    fseek(fp,-i,SEEK_END);    /* i=1时，定位于字母z */  
    putchar(fgetc(fp));      /* 显示读出的字符，位置指针+1 */  
}  
  
fclose(fp);  
}
```

案例分析

问题

- ✿ 建立一个文件，向其中写入一组学生的姓名和成绩，然后从该文件中读出成绩大于80分的学生信息，并显示在屏幕上。

分析

- ✿ 先用fopen()函数建立文件，再用fwrite()函数写入学生记录。
- ✿ 用rewind()函数定位于文件开头，用fread()函数从文件中顺序读出一个个记录，并判断成绩是否大于80分。若是，则输出。
- ✿ 在读记录之前，要用fseek()函数定位在正确的位置上。

案例分析

源代码 (cw1604.c)

```
#include <stdio.h>
#include <stdlib.h>

void main( )
{
    FILE *fp;
    int i;

    struct student {
        char name[10];
        int score;
    } s, stud[]={{"张雄", 90},
                 {"李平", 72},
                 {"孙兵", 80},
                 {"刘军", 88},
                 {"王伟", 92}};
```

案例分析

源代码

```
if((fp=fopen("stud.bin","wb+"))==NULL) {  
    printf("不能建立文件stud.bin\n");  
    exit(0);  
}  
  
for(i=0;i<5;i++)  
    if(fwrite(&stud[i],sizeof(struct student),1,fp)!=1)  
        printf("写文件错误");
```

案例分析

源代码

```
rewind(fp);

printf("姓名      成绩\n");
printf("-----\n");
for(i=0;i<5;i++) {
    fseek(fp,i*sizeof(struct student),SEEK_SET);
    fread(&s, sizeof(struct student),1,fp);
    if (s.score>80)
        printf("%s      %d\n",s.name,s.score);
}

fclose(fp);
}
```

小结

- 要存取文件，需要创建一个文件指针（类型为FILE*）并将其和一个具体的文件名关联起来。后续的代码就通过该指针来处理文件。
- 系统自动创建输入输出缓冲区以加快数据传输的速度。
- C如何理解关联文件的输入流和输出流取决于所使用的输入输出函数。
 - ✿ 字符方式：fgetc(), fputc(), fscanf(), fprintf()
 - 将读到的每个字节理解为一个字符编码。
 - fscanf()函数将字符编码转换成格式说明符指定的数据类型。
 - 输出时把非字符数据转换成字符表示以后再将其放到输出流。
 - ✿ 二进制方式：fread(), fwrite()

小结

可以采取两种方式读写文件：

✿ 顺序方式

- 从头到尾顺序读取文件中的数据。

✿ 随机方式

- **fseek()**函数和**ftell()**函数允许程序移动位置指针到文件中的任意位置进行随机读写。